WIF3009
Python for Scientific Computing

Introduction to Python (Part 1)

Python History

- Python is a really powerful high-level, object-oriented programming language.
- It is created by Guido van Rossum, a Dutch programmer.
- The idea for Python came when Van Rossum was bored and looking for a “hobby programming project that would keep him occupied during Christmas as his office was closed”.

Photo of the man who created a programming language because his office was closed for Christmas.

Python History

- Python was first released in **February 1991.**
- It is a **general purpose** language i.e. it has a wide range of applications from web development (Django), scientific and mathematical computing (NumPy) to desktop GUI (Pygame)
- The idea for Python came when Van Rossum was bored and looking for a “hobby programming project that would keep him occupied during Christmas as his office was closed”.
- No. Python is not named after the snake.
- The name Python was adopted from the **British comedy series Monty Python.**

Features of Python

- Python has a simple and elegant syntax: a one-liner code should be a common thing in Python!
- It is not overly strict: Python automatically detects the data types. You do **not need to add semicolons (;)** at the end of every statement, the nightmare for every programmer.
- It is highly expressive: human-readable and understandable.
- Python has **great community** and **support**: it is open source!

Features of Python

Python Distribution

- Python can be downloaded via web (python.org).
- This course will utilize **Python 3.11**
- As a better alternative, setting up Python is best done using Anaconda package.
- **Anaconda/Miniconda** is a free and open source distribution of Python and R for data science and machine learning related applications; simplifies package management and deployment.

Integrated Development Environment (IDE)

- Like any other programming languages, Python has a number of IDEs in range:
  - **PyCharm**
  - **Spyder**
  - **PyDev**
  - **etc.**
- In this course, we will be using **JupyterLab**. It is a web based notebook for interactive and reproducible computing/programming.

IDE - JupyerLab

- **JupyterLab** provides IDE-like environment for interactive programming.
- The Notebook allows you to **program and individually** run each line of your code for immediate output.
- You can also **create markdowns in** the notebook for better reporting and reproducibility.
- The **notebook** file (.ipynb) can be easily transformed into a **full Python script file** (.py).

Entering Input

- You can program in Python using two ways:
  - Python console
  - Python script
  - Python notebook
- Python console is useful for quick analysis or if you want to run a short code.
- In the Python console, you can quickly type out:
  2+3 for Python to return the answer **5.**

```
[1]: 2+3

[1]: 5
```

Variables and Type

- You can also store values inside variables.
- Variables are specific, case-sensitive names.
- You can call up a value through a variable name.
- Storing values inside variables encourage reproducibility.
- Hence, any time you wish to alter the values, you only need to edit it via the variable.

```
[2]: height = 1.76

[3]: weight = 55

[4]: height

[4]: 1.76
```

Variables and Type

```
[7]: height = 1.76
     weight = 55
     bmi = weight / height ** 2

     print(bmi)

     17.75568181818182
```

- You can carry out any operations on variables just like you would on normal values.
- As a side note, Python accepts normal mathematical operation e.g. division (/), add (+), minus (-), multiply (*), exponent (**)
- The figure above shows an example to calculate BMI.
- To include comments, precede the statement with #. This line will be ignored.

Variables and Type

- You would have noticed by now that we did not explicitly define any data types when creating the variables.
- Python automatically detects the data type based on what data is passed in.
- You can use the function `type` to check the data type of your variable.

```
[8]: type(bmi)
[8]: float
[9]: day_of_week = 5
[10]: type(day_of_week)
[10]: int
[11]: surname = 'hanif'
[12]: type(surname)
[12]: str
```

Variables and Type

- It is really important to note that each of Python types comes with its own behavior.
- For instance, you can create a string (str) by using either single or double quotes.
  - ‘pencil’ and “pencil” both creates a string.
- Besides that, different type produces different output although subjected to a same operation.

```
[13]: 2+3

[13]: 5

[14]: 'hazim' + 'hanif'

[14]: 'hazimhanif'
```

Printing in Python

- When using Python in terminal or console, the output automatically gets printed.
- However, more often than not, we will be writing Python scripts that require printing some output.
- Python provides multiple ways of printing depending on your objective and style.

```
[15]: first_name = 'hazim'
      last_name = 'hanif'

[16]: print(first_name + ' ' + last_name)
      hazim hanif

[18]: print('{} {}'.format(first_name, last_name))
      hazim hanif

[19]: print(f'{first_name} {last_name}')
      hazim hanif
```

Comments

- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.

```
[189]: #This is a comment

       print("Hello WIF3009!")

       Hello WIF3009!
```

Data Types

- Python has **five** standard data types:
  - Numbers
  - String
  - List
  - Boolean
  - Dictionary
- As briefly mentioned in previous slides, Python sets the variable type based on the value that is assigned to it.
- Unlike other languages, Python intelligently changes the variable type if the variable value is set to another value.

```
[22]: var = 123
      type(var)

[22]: int

[23]: var = 'john'
      type(var)

[23]: str
```

Numbers

- Python numbers are created by the standard Python method.
- Common number types are `int` and `float`.
- `int` or integers are whole numbers while floats are real values.
- You can force Python to switch the types by using type casting
  - To convert integer 12 to float: float(12)
  - To convert float 12.52 to integer: int(12.52)
- When dividing numbers, a single slash (/) returns a float while double slash (//) returns an integer.

```
[24]: 50/3

[24]: 16.666666666666668

[25]: 50//3

[25]: 16
```

String

- In Python, strings are created by enclosing characters in quotes.
- Previous slides have mentioned that Python uses single quotes (‘) or double quotes (“) to create strings.
- However, Python also accepts triple quotes (‘’’) to denote literal strings.
- The triple quoted strings will automatically continue across the end of line statement (multiline strings)

```
[29]: message = '''This is a string with multiple line.
      It is true.
      Trust me. watch this'''

[31]: print(message)

      This is a string with multiple line.
      It is true.
      Trust me. watch this
```

String

- Strings can be accessed as a whole string or sliced as a substring using square brackets '[ ]'.
- This is what we called as *SLICING*.
- Notice that the slicing syntax [0:5] will start slicing from character 0 up until 4. Character 5 is not inclusive.

```
[33]: myvar = "Hello World"
      myvar[0:5]

[33]: 'Hello'

[34]: myvar[:5]

[34]: 'Hello'

[36]: myvar[6:]

[36]: 'World'

[37]: myvar[6:-1]

[37]: 'Worl'
```

String

- Python can use a special syntax, the string format function to print multiple strings/numbers.

```
[39]: name = 'John'
      age = 24

[40]: print('{} is {} years old.'.format(name, age))

      John is 24 years old.
```

Booleans

- Booleans are **True** or **False** values. It can also be represented as **1** or **0** respectively.
- Booleans are useful when constructing conditional statements.
- Python automatically detects a Boolean value if you assign True or False to a variable.

```
[83]: var = True

[84]: type(var)

[84]: bool
```

Storing collections of data in Python

- Natively, there are 4 built-in data types in Python that is used to store collections of data
  - List
  - Dictionary
  - Tuple
  - Set

- We will go into each one of these.

Lists

- Previously, each variable represents a single value, separated by comma (,)
- However, data science will always deal with many data points.
- For example, it is inconvenient to store the heights of the entire family in separate variables.
- With list, you can store all the values in a single data structure.

```
[43]: heights = [1.24, 1.67, 1.98]
      print(heights)

      [1.24, 1.67, 1.98]

[44]: type(heights)

[44]: list
```

Lists

- You can **subset the list** to extract the preferred values.
- Python uses zero-based indexing. In other words, all elements in Python is counted starting from 0.

```
[58]: heights = ['Abu', 'Wong','Bryan']
                  0       1       2
```

- Therefore, to select *Wong*, you type `heights[1]`.
- You can also subset from the rear by using negative index. To select *Bryan* , you can do `heights[-1]`.

```
[63]: heights[0]

[63]:  'Abu'
```

```
[61]: heights[1]

[61]:  'Wong'
```

```
[62]: heights[-1]

[62]:  'Bryan'
```

Lists

- List can contain any data types. It can even hold different types in a single list.
- It can even hold lists in a list!

```
[45]: heights = ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98]
      print(heights)

      ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98]
```

List containing different types (strings and floats)

Lists

- List with multiple list.
- To access a list, you can just put in the index location of the element that you want to access in the bracket [ ][ ].

```
[52]: heights = [['Abu', 1.24], ['Wong', 1.67], ['Bryan', 1.98]]
      print(heights)

      [['Abu', 1.24], ['Wong', 1.67], ['Bryan', 1.98]]

[53]: heights[0]

[53]: ['Abu', 1.24]

[54]: heights[0][0]

[54]: 'Abu'
```

Lists

- What if you want to extract values that are not in a continuous range? In other words, what if you want to select only elements 1,3 and 5.
- While it is easier using Python NumPy (discussed later), you can still achieve this objective via the following:

```
[69]: heights = ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98]

      list(heights[i] for i in [1,3,5])

[69]: [1.24, 1.67, 1.98]
```

- You are starting to see the power and elegance of a Python code!

Lists

- To change the list elements, you simply assign a new value to the element. For example: `heights[0] = 'Kamarul'` will replace the existing name, Abu with Kamarul.
- Similarly, you can change elements within a range `heights[0:2] = ['Kamarul', 1.88]`
- You can add or append to the list using three ways.
  - Concatenate using +
  - Using `append()` method
  - Using `extend()` method

Lists

- Concatenating using the + sign is the simplest approach.

```
[71]: heights

[71]: ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98]

[70]: heights + ['Razak', 1.77]

[70]: ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98, 'Razak', 1.77]

[72]: heights

[72]: ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98]
```

- But the changes will not persist unless you assign it back to itself.

```
heights = heights + ['Razak', 1.77]
heights

['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98, 'Razak', 1.77]
```

Lists

- To add just a single value, you can use the `append()` method.

```
[75]: heights

[75]: ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98]

[78]: heights.append('Razak')
      heights.append(1.77)
      heights

[78]: ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98, 'Razak', 1.77]
```

Lists

- To add multiple values, use the `extend()` method.

```
[81]: heights

[81]: ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98]

[82]: heights.extend(['Razak',1.77])
      heights

[82]: ['Abu', 1.24, 'Wong', 1.67, 'Bryan', 1.98, 'Razak', 1.77]
```

Lists

- To delete elements from a list, use the `del()` function.
- For example: `del(heights[0:2])`.
- It is worth noting that when creating a variable, Python actually creates an address that points to a memory location.
- To prove this, if we create another variable `heights_b` and equate it to `heights`, Python does not actually create another new list. In fact, both of the variables are now pointing to the same address.
- As a result, if you alter the values in `heights_b`, it will also change the values in `heights`.

Lists

- Therefore, if you wish to create a literal copy of a list, you should do

```
heights_b = list(heights)
```

- Now, when you alter value in `heights_b`, it will no longer affect the original `heights` list because both of these variables are not sharing the same memory location.
- Besides that, Python lists do not support element wise operations.
- For example, if we want to calculate the BMI from two lists: `heights/weights**2` will yield an error.

Dictionaries

- Let's say we have 2 separate lists containing names and heights. What if you want Wong's height?

- It is less intuitive and inconvenient as you need to find the index before able to extract the height.

```
[86]: names = ['Abu', 'Wong', 'Bryan']

      heights = [1.24, 1.67, 1.98]

      index_abu = names.index('Wong')
      index_abu

[86]: 1

[87]: heights[index_abu]

[87]: 1.67
```

Dictionaries

- As a better alternative, we can store the names and heights in a dictionary.
- Dictionary in Python is a collection of **key-value pairs** enclosed between curly brackets { }.
- The keys and values are separated by a colon. You can quickly extract the value by specifying the key.

```
[89]: heights_dict = {'Abu':1.24, 'Wong':1.67, 'Bryan':1.98}
      heights_dict

[89]: {'Abu': 1.24, 'Wong': 1.67, 'Bryan': 1.98}

[90]: heights_dict['Bryan']

[90]: 1.98
```

Dictionaries

- There are some functions such as pop, get, ...

```
[97]: heights_dict

[97]: {'Abu': 1.24, 'Wong': 1.67, 'Bryan': 1.98}

[93]: heights_dict.pop('Abu')

[93]: 1.24

[94]: heights_dict

[94]: {'Wong': 1.67, 'Bryan': 1.98}
```

Dictionaries

List	Dictionary
Select, update and remove: [ ]	Select, update and remove: [ ]
Indexed by range of numbers	Indexed by unique keys
Collection of values Order matters	Lookup table with unique keys Order does not matter

Tuple

- Tuples are used to store multiple items in a single variable.
- A tuple is a collection which is ordered and **unchangeable**.
- Tuple items are **ordered**, **unchangeable**, and **allow duplicate values**.
- Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

```
[99]: mytuple = ("apple", "banana", "cherry")
      print(mytuple)

      ('apple', 'banana', 'cherry')

[101]: mytuple2 = ("chicken", "fox")
       print(mytuple2)

       ('chicken', 'fox')
```

Sets

- Sets are used to store multiple items in a single variable.
- A set is a collection which is ***unordered***, ***unchangeable****, ***unindexed*** , ***no duplicates.***
- But you can remove items and add new items.
- Sets are written with curly brackets. { } or calling *set([ ]) or set(( ))* .

```
[106]: myset = {"apple", "banana", "cherry"}
       print(myset)

       {'apple', 'banana', 'cherry'}

[108]: myset2 = set(["apple", "banana", "cherry","banana"])
       print(myset2)

       {'apple', 'banana', 'cherry'}
```

Set

- With sets, you can do intersection, union, etc. Pretty much like the “set” in a Venn diagram.
- Just remember, no duplicates.

```
[112]: myset = {"apple", "banana", "cherry"}
       myset2 = {"lemon", "banana"}

[113]: myset.intersection(myset2)

[113]: {'banana'}

[114]: myset.difference(myset2)

[114]: {'apple', 'cherry'}

[118]: myset2.difference(myset)

[118]: {'lemon'}

[119]: myset.union(myset2)

[119]: {'apple', 'banana', 'cherry', 'lemon'}
```

Operators

- Operators are special symbols in Python that carry out arithmetic or logical computation.
- For example the + symbol carry out arithmetic operation as in 2 + 5 = 7.
- The different types of operators in Python are:
  - Arithmetic operator
  - Comparison operator
  - Logical operator
  - Identity operator
  - Membership operator

Operators (arithmetic)

- Arithmetic operators carry out mathematical operations like addition, subtraction etc.

```
[120]: x=2
       y=3

[121]: x+y

[121]: 5

[122]: y-x

[122]: 1

[123]: x*y

[123]: 6
```

Operators (comparison)

- Comparison operators compare between values and return `True` or `False` according to the condition.

```
[132]: x == 2
[132]: True
[133]: x == 4
[133]: False
[134]: x != y
[134]: True
[135]: x > y
[135]: False
[136]: x <= y
[136]: True
```

Operators (logical)

- Logical operators are the `and`, `or` and `not` operators.
- `and` returns `True` if both operands are true.
- `or` returns `True` if either operands are true.
- `not` returns `True` if operand is false.

```
[151]: x and y
[151]: False

[152]: x or y
[152]: True

[153]: not x
[153]: False
```

Operators (identity)

- Identity operators check if two values occupy the same part of the memory.

```
[148]: x = True
       y = False

[149]: x is y

[149]: False

[150]: x is not y

[150]: True
```

Operators (membership)

- Membership operators check if a value exist in a sequence (i.e., string, list, tuple, set and dictionary)

```
[154]: name = 'john doe'

[156]: 'john' in name

[156]: True

[158]: simpson = ['mmmmm','donuts']

[160]: 'donuts' in simpson

[160]: True

[161]: 'ciken' in simpson

[161]: False
```

Conditional Statement (if - else)

- Python supports the usual logical conditions from mathematics:
  - Equals: a == b
  - Not Equals: a != b
  - Less than: a < b
  - Less than or equal to: a <= b
  - Greater than: a > b
  - Greater than or equal to: a >= b
- These conditions can be used in several ways, most commonly in "if statements" and loops.
- An "if statement" is written by using the if keyword.

Conditional Statement (if – else)

if condition:

    expression

```
[163]: x = 3

       if x == 3:
           print(True)

       True
```

if condition:

    expression

else:

    expression

```
[165]: x = 3

       if x == 4:
           print(True)
       else:
           print(False)

       False
```

Conditional Statement (if – elif – else)

- The elif keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

```
[167]: x = 3

       if x == 4:
           print(True)
       elif x == 3:
           print('MIDDLE')
       else:
           print(False)

       MIDDLE
```

Conditional statement (multiple conditions)

- Multiple conditions using ‘or’
  - Follow the OR truth table

```
[168]: x = 3

       if (x == 4) or (x == 3):
           print('IN')
       else:
           print('OUT')

       IN
```

- Multiple conditions using ‘and’
  - Follow the AND truth table

```
[169]: x = 3

       if (x == 4) and (x == 3):
           print('IN')
       else:
           print('OUT')

       OUT
```

Loops (for)

- `for` loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- With the `for` loop we can execute a set of statements, once for each item in a list, tuple, set etc.

```
[171]: family = ['John', 'Brian', 'Steph']

       for siblings in family:
           print(siblings)

       John
       Brian
       Steph
```

Loops (for)

- You can also use `for` loop to print the keys and values in a dictionary by using the items() method.

```
[181]: bmi.items()

[181]: dict_items([('John', 23), ('Brian', 26), ('Steph', 21)])
```

```
bmi = {'John':23, 'Brian':26, 'Steph':21}

for key,value in bmi.items():
    print('%s : %d' % (key,value))

John : 23
Brian : 26
Steph : 21
```

Loops (while)

- With the `while` loop we can execute a set of statements as long as a condition is **true**.

```
[184]: i = 0
       while i < 5:
         print(i)
         i += 1

       0
       1
       2
       3
       4
```

```
[185]: i = 5
       while i >= 0:
         print(i)
         i -= 1

       5
       4
       3
       2
       1
       0
```

To be continued…

Indentation checking in Python is like: